



R POUR L'ANALYSE DE DONNÉES

R MARKDOWN, TESTS STATISTIQUES, MACHINE LEARNING

Ressources et cours consultés

- <https://rmarkdown.rstudio.com/>
- <https://larmarange.github.io/analyse-R/rmarkdown-les-rapports-automatistes.html>
- https://epirhandbook.com/fr/new_pages/stat_tests.fr.html
- <https://www.math.univ-toulouse.fr/~besse/Wikistat/pdf/st-l-inf-tests.pdf>
- http://www.thibault.laurent.free.fr/cours/R_intro/chapitre5_tests.html

Table des matières

1	Introduction à R Markdown	3
1.1	Installer et charger le package <code>rmarkdown</code>	3
1.2	Créer un premier document R Markdown	3
1.3	Générer le document (Bouton Knit)	5
2	Structure d'un fichier R Markdown (.Rmd)	7
2.1	Le langage Markdown	8
2.2	Titres	8
2.3	Paragrophes et Mise en valeur	8
2.4	Liens et Images	8
2.5	Séparateurs / Ligne horizontale	9
2.6	Bloc de citation	9
2.7	Listes	9
2.8	Tableaux	10
3	Code en R Markdown	10
4	Comment choisir un test statistique ?	12
4.1	Test du Khi-deux d'indépendance	12
4.1.1	Conditions d'utilisation	12
4.1.2	Étapes	12
4.1.3	Exemple :	13
4.1.4	Test d'indépendance (Khi-deux) et taille d'effet	18
4.2	Corrélation entre deux variables quantitatives	19
4.2.1	Analyse univariée (quantitatif)	19

4.2.2	Corrélation (bivarié) : Pearson, Spearman, Kendall	23
5	Variable qualitative $\leftrightarrow$ variable quantitative	24
5.1	Exemple 1 : 2 groupes (mpg selon am : automatique vs manuel)	25
5.2	Exemple 2 : > 2 groupes (mpg selon cyl : 4, 6, 8 cylindres)	25
6	Modélisation	27
6.1	Régression linéaire simple	27
6.2	Régression linéaire multiple	27
6.3	Régression logistique	27

1 Introduction à R Markdown

L'extension `rmarkdown` permet de générer des documents **dynamiques** en combinant du texte mis en forme et des résultats produits par du code R. Les documents peuvent être exportés en **HTML**, **PDF** ou **Word**. C'est un outil essentiel pour la **communication** et la **reproductibilité** des analyses.

1.1 Installer et charger le package `rmarkdown`

- **Installer** un package (`install.packages()`) : à faire *une seule fois* sur une machine.
- **Charger** un package (`library()`) : à faire *à chaque nouvelle session* où vous l'utilisez.

1) Installation et chargement R Markdown

```
1 #installation
2 install.packages("rmarkdown")
3
4 #chargement
5 library(rmarkdown)
```

R installe automatiquement les **dépendances** nécessaires, vous n'avez pas à les lister vous-même. Le chargement est à répéter à chaque fois que vous ouvrez une nouvelle session R (ou RStudio) et que vous souhaitez utiliser `rmarkdown`.

3) Vérifier que tout fonctionne

```
1 # Vérifier la version installée (optionnel)
2 packageVersion("rmarkdown")
3
4 # Aide rapide (ouvre la page d'aide dans R)
5 help(package = "rmarkdown")
```

1.2 Créer un premier document R Markdown

Pour créer un premier rapport dynamique :

1. Ouvrir RStudio.
2. Cliquer sur **File > New File > R Markdown...**

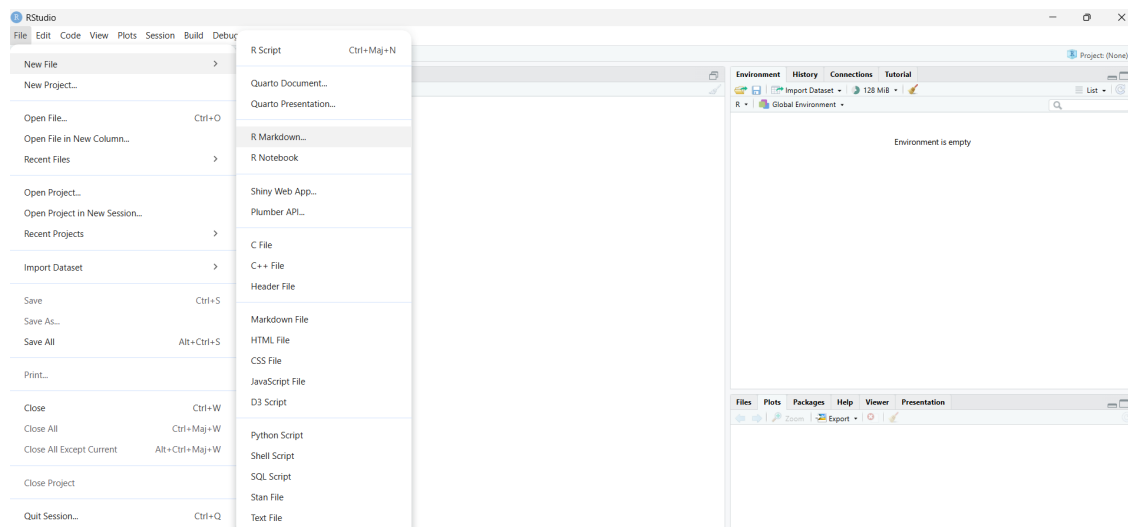


FIGURE 1 – Accéder au menu de création d'un fichier R Markdown dans RStudio.

Une fenêtre s'ouvre alors. Elle permet de définir les informations générales du document.

3. Saisir un **titre** et un **auteur**.
4. Choisir le **format de sortie** (HTML par défaut).
5. Cliquer sur **OK**.

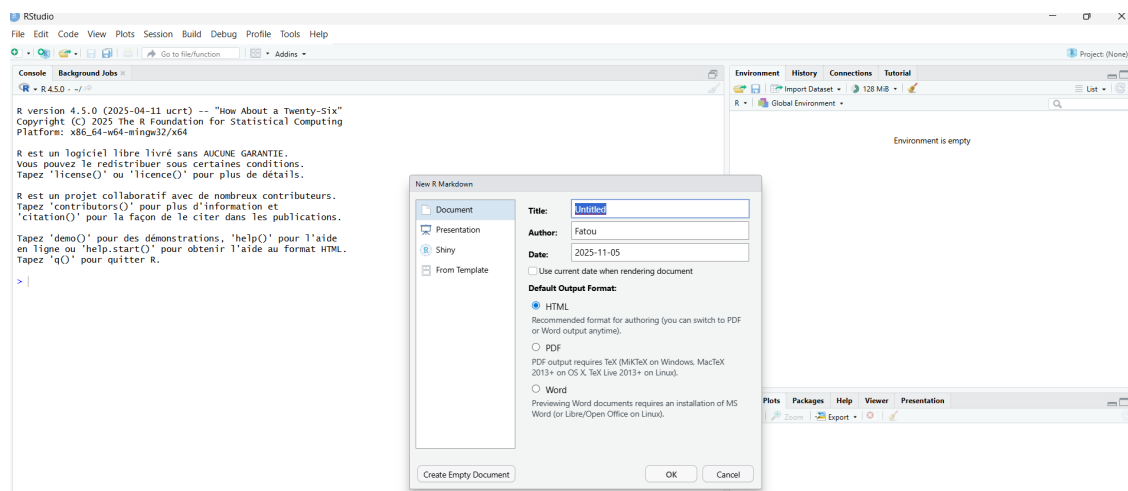


FIGURE 2 – Fenêtre de configuration du document : titre, auteur et format de sortie.

RStudio crée automatiquement un fichier `.Rmd` contenant une structure minimale prête à être modifiée.

6. Enregistrer le fichier dans un dossier de travail.
7. Cliquer sur le bouton **Knit** pour générer le rapport.

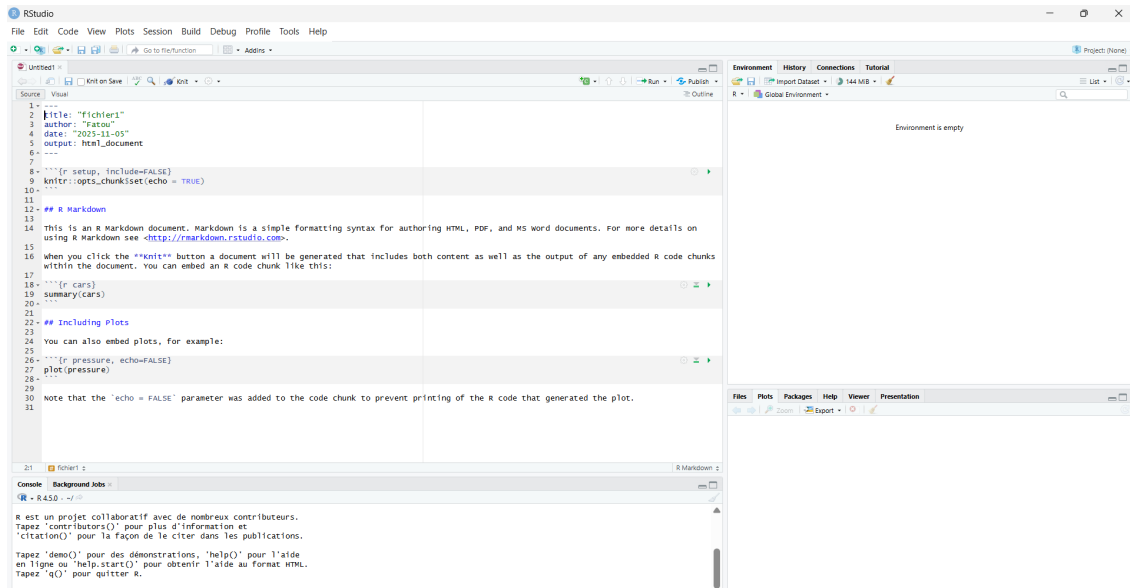


FIGURE 3 – R Markdown généré automatiquement avec en-tête YAML et code d'exemple.

1.3 Générer le document (Bouton Knit)

Une fois le fichier `.Rmd` créé et éventuellement modifié, il faut **l'enregistrer** dans un dossier de votre ordinateur :

- Cliquez sur **File > Save As...**
- Choisissez un dossier de travail
- Donnez un nom au fichier, par exemple : `fichier_1.Rmd`

Pour produire le document final (HTML, PDF ou Word), il suffit ensuite de cliquer sur le bouton : 

Le bouton **Knit** :

- exécute automatiquement tous les blocs de code R présents dans le document,
- puis assemble le texte et les résultats,
- pour générer un rapport prêt à être partagé.

Il permet également de **choisir le format de sortie du document**. En cliquant sur la petite flèche à côté de **Knit** :

Après avoir choisi le format de sortie puis cliqué sur **Knit**, RStudio génère automatiquement le rapport final. Voici le document obtenu au format **HTML** :

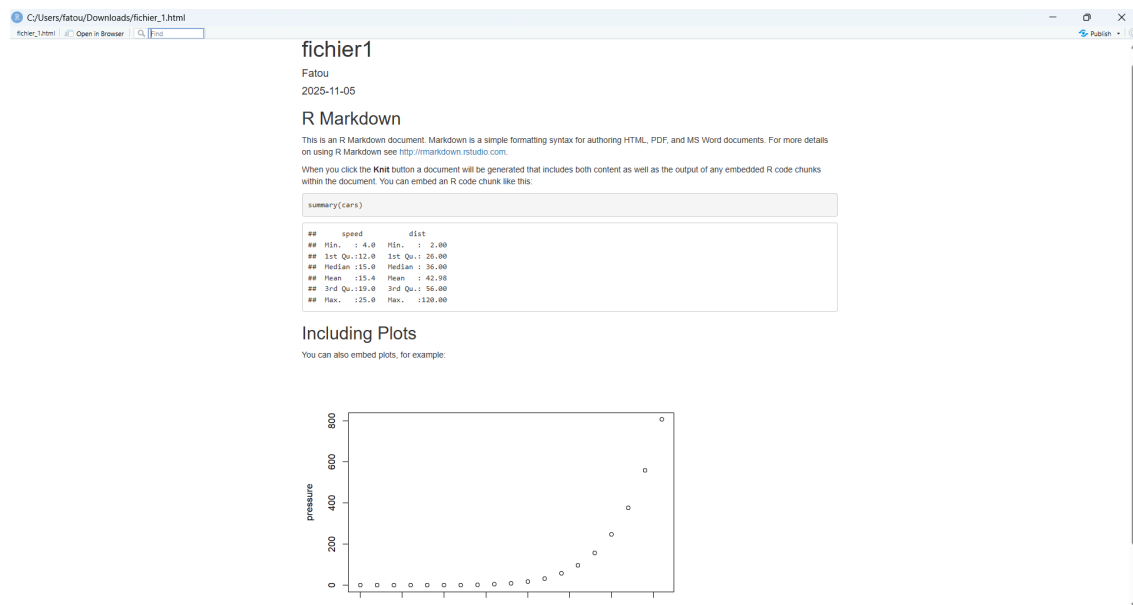


FIGURE 4 – Exemple de rapport HTML généré à partir d’un fichier R Markdown.

Le document au format **Word**, ce qui peut être pratique pour modifier le rapport :

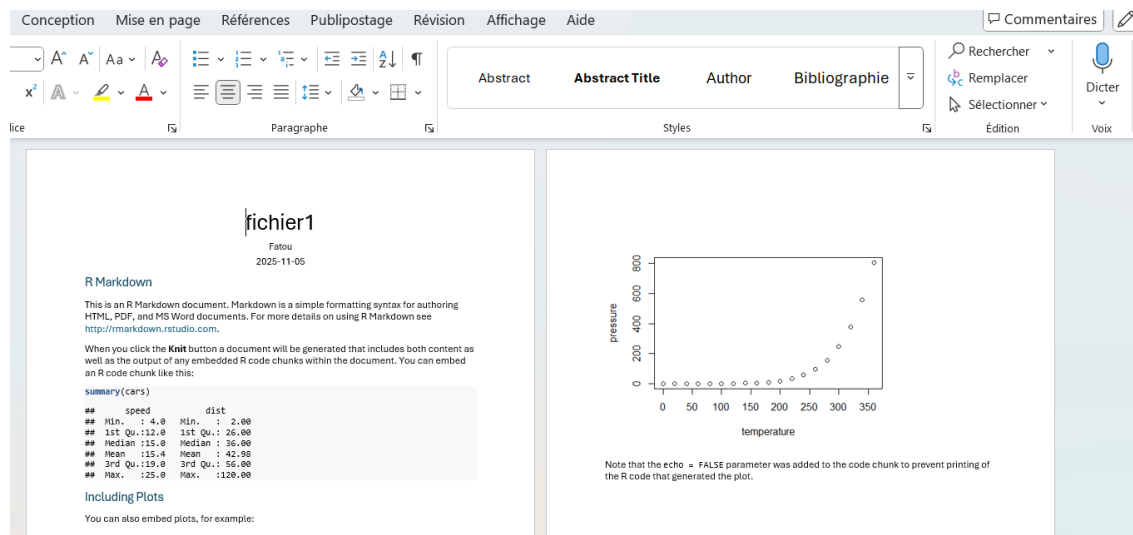


FIGURE 5 – Exemple de rapport au format Word généré avec R Markdown.

Dans les deux cas, le document final contient :

- du **texte explicatif** (vos commentaires, interprétations, descriptions),
- les **résultats des commandes R** (tableaux, statistiques, résumés),
- ainsi que les **graphiques et visualisations** générés dans le fichier.

L’intérêt de R Markdown est donc de réunir dans un seul document :

Code → Résultats → Interprétation

2 Structure d'un fichier R Markdown (.Rmd)

Un fichier .Rmd contient généralement trois types d'éléments :

- un **en-tête YAML** (métadonnées) entouré par --- ;
- du **texte** en Markdown (titres, paragraphes, listes, explications) ;
- des **blocs de code R** appelés *chunks*, délimités par ````${r}```` et ````\n`.

Le contenu du fichier :

```
1 #en-tête
2 ---
3 title: "fichier1"
4 author: "Fatou"
5 date: "2025-11-05"
6 output: html_document
7 ---
8 #bloc code R
9 ```{r setup, include=FALSE}
10 knitr::opts_chunk$set(echo = TRUE)
11 ```
12 #texte
13 ## R Markdown
14 This is an R Markdown document. Markdown is a simple formatting syntax for
    authoring HTML, PDF, and MS Word documents. For more details on using R
    Markdown see <http://rmarkdown.rstudio.com>.
15
16 When you click the **Knit** button a document will be generated that includes both
    content as well as the output of any embedded R code chunks within the document
    . You can embed an R code chunk like this:
17
18 #code R
19 ```{r cars}
20 summary(cars)
21 ```
22 #texte
23 ## Including Plots
24
25 You can also embed plots, for example:
26 #code R
27 ```{r pressure, echo=FALSE}
28 plot(pressure)
29 ```
30 #texte
31 Note that the `echo = FALSE` parameter was added to the code chunk to prevent
    printing of the R code that generated the plot.
```


2.1 Le langage Markdown

Il est volontairement **simple, lisible et rapide à écrire**.

2.2 Titres

Les titres se créent en ajoutant des `#` au début de la ligne :

```
1 # Titre de niveau 1
2 ## Titre de niveau 2
3 ### Titre de niveau 3
```

Il existe aussi une autre syntaxe pour les niveaux 1 et 2 :

```
1 Titre de niveau 1
2 =====
3
4 Titre de niveau 2
5 -----
```

2.3 Paragraphes et Mise en valeur

Un simple retour à la ligne ne crée pas un nouveau paragraphe : il faut laisser une ligne vide entre deux paragraphes.

Pour mettre en forme du texte :

- **Gras** : `**texte**` ou `__texte__`
- *Italique* : `*texte*` ou `_texte_`

2.4 Liens et Images

Liens simples :

```
1 https://rmarkdown.rstudio.com
```

Lien cliquable avec texte :

```
1 [Aller vers le site R Markdown](https://rmarkdown.rstudio.com)
```

Insérer une image :

```
1 ![Texte alternatif](chemin/vers/image.png "Description facultative")
```

2.5 Séparateurs / Ligne horizontale

```
1 #Pour séparer visuellement des parties :  
2 ---  
3 ***  
4 * * *  
5 - - -
```

2.6 Bloc de citation

On peut aussi créer des blocs de citations (équivalent à la balise blockquote), en commençant les lignes par ">".

```
1 > Ceci est un bloc de citation.  
2 > Il permet de mettre en avant une information importante.
```

2.7 Listes

Il est possible de créer deux types de listes (non-ordonnées et ordonnées). Il faut noter cependant que les listes doivent être séparées des paragraphes par (au moins) une ligne au-dessus et en-dessous. De plus, pour les sous-listes, il faut mettre une tabulation (ou deux si sous-sous-liste, et ainsi de suite).

Liste non ordonnée : Ce sont les plus simples à écrire. Les items commencent tous par le même caractère. Le caractère utilisé ici ("-") peut être remplacé par "*" ou "+".

```
1 - Élément A  
2 - Élément B  
3   - Sous-élément B1  
4   - Sous-élément B2
```

Liste ordonnée : Pour avoir une liste ordonnée, il est nécessaire de commencer la ligne par un chiffre suivi de ".". Par contre, le chiffre indiqué n'est pas pris en compte. Ce qui fait que la sous-liste ici produira bien 1, 2 et 3.

```
1 1. Premier point  
2 2. Deuxième point  
3   1. Sous-point  
4   2. Sous-point
```

2.8 Tableaux

On peut définir un tableau directement, en précisant d'abord le nom des colonnes (séparés par " "). Il faut ensuite ajouter une ligne de séparation avec --- (au moins) pour chaque colonne, et toujours séparer les colonnes par " " dans toutes les lignes. Puis, les valeurs du tableau se placent ligne par ligne, également séparées par " ".

Le style (alignement) des colonnes peut être contrôlé dans la ligne de séparation :

- --- ou :-- : aligné à gauche (par défaut)
- --: : aligné à droite
- :-: : centré

3 Code en R Markdown

L'intérêt de ce langage est aussi de pouvoir intégrer des éléments de code (soit des blocs complets, soit des éléments en ligne), permettant de présenter le travail fait.

Pour créer un bloc de code, on l'encadre avec trois accents graves (appelés *backticks*) avant et après.

Il est également possible de créer un bloc de code en indentant les lignes avec une tabulation ou quatre espaces.

Chunk Un chunk sera donc un bloc de commande R (ou autre langage possible) qui sera exécuté par R Studio. Pour cela, il faut indiquer sur la première ligne le langage utilisé. Pour R, voici donc un exemple simple

Il est possible de nommer le chunk en lui donnant un label (sans espace, sans accent) après r dans les . Ceci est intéressant surtout dans l'étape de développement, car si une erreur arrive lors de l'exécution, il sera plus facile de retrouver dans quel chunk est l'erreur (indiqué lors de l'affichage de l'erreur).

De plus, il est possible de mettre des options dans le chunk, toujours dans les , après une ", ". Voici quelques options classiques et utiles (avec leur valeur par défaut indiquée, si elle existe) :

include = TRUE : si FALSE, le code est exécuté mais il n'est pas inclus dans le document (ni le code, ni son résultat) echo = TRUE : si FALSE, le code n'est pas affiché mais bien exécuté eval = TRUE : si FALSE, le code est affiché mais n'est pas exécuté results = 'markup' : permet de définir comment le résultat est affiché (intéressant pour les tableaux, cf plus loin) fig.cap : titre du graphique produit Il est possible de mettre plusieurs options, toutes séparées par des ", ".

```

1  ```{r}
2  # Création d'un vecteur
3  notes <- c(12, 15, 17, 10)
4
5  # Calcul de la moyenne
6  mean(notes)
7  ```

```

Tests statistiques

Quand on observe un phénomène au hasard (par exemple, en tirant un échantillon), il est difficile de savoir si ce que l'on voit est dû à une **vraie différence** ou simplement au **hasard**.

Un **test statistique** est une méthode qui sert à répondre à cette question. Il permet de vérifier si l'effet observé dans les données est suffisamment important pour penser qu'il existe réellement dans la population étudiée, et pas seulement par hasard.

Faire un test statistique consiste à :

- poser une **hypothèse nulle** (H_0) : souvent « pas de différence » ou « pas de relation » ;
- poser une **hypothèse alternative** (H_1) : c'est l'effet que l'on cherche à montrer ;
- calculer une **statistique de test** à partir des données, puis une **p-valeur** ;
- décider : si la p-valeur est petite ($< 0,05$), on rejette H_0 et on conclut qu'il existe bien un effet significatif.

En résumé, un test statistique est un outil qui aide à décider si ce que l'on observe dans un échantillon reflète une **vraie tendance dans la population** ou seulement un **effet du hasard**.

Avant de réaliser un test, il est indispensable de passer par une **analyse descriptive** :

- **Univariée** : décrire chaque variable séparément (fréquences, histogrammes, etc.).
- **Bivariée ou multivariée** : explorer les relations entre variables.
 - Deux variables qualitatives : tableau de contingence, graphiques en barres, etc.
 - Une variable qualitative et une variable quantitative : comparaison des moyennes, boxplots.
 - Deux variables quantitatives : nuage de points, calcul de corrélation.

Ces étapes exploratoires permettent de formuler des hypothèses. Les **tests statistiques** servent ensuite à vérifier si les différences ou relations observées sont significatives ou dues au hasard.

4 Comment choisir un test statistique ?

Le choix d'un test dépend de la **nature des variables** étudiées.

- Deux variables **qualitatives** : test du χ^2 d'indépendance, test exact de Fisher.
- Une variable **qualitative** et une variable **quantitative** : test t de Student (2 groupes), ANOVA (plus de 2 groupes), ou tests non paramétriques (Wilcoxon, Kruskal-Wallis).
- Deux variables **quantitatives** : corrélation de Pearson (relation linéaire, normalité supposée), ou corrélation de Spearman si non-normalité.

D'autres tests existent également, vous pouvez consulter les liens proposés dans la page de ressources de la première partie pour approfondir ou découvrir d'autres possibilités.

4.1 Test du Khi-deux d'indépendance

Teste l'existence d'une association entre *deux variables qualitatives*.

H_0 : les variables sont indépendantes. H_1 : les variables sont associées (dépendantes).

4.1.1 Conditions d'utilisation

- Observations indépendantes.
- Effectifs *attendus* suffisamment grands : règle pratique ≥ 5 dans au moins 80 % des cellules.
- Si l'effectif est petit, préférer le **test exact de Fisher**

4.1.2 Étapes

1. **Analyse univariée** : décrire séparément chaque variable (fréquences, barplots...).
2. **Analyse bivariée** : tableau de contingence, proportions par ligne/colonne...
3. **Test de Khi-deux** : lancer le test, lire la p -value, examiner les *résidus standardisés*.
4. **Taille d'effet** : mesurer l'intensité du lien avec le V de Cramér.

Taille d'effet

$$V_{\text{Cramér}} = \sqrt{\frac{\chi^2}{n(\min(r, c) - 1)}} \in [0, 1]$$

Lecture de V (repères courants) : ≈ 0 négligeable ; 0.1 faible ; 0.3 modérée ; ≥ 0.5 forte.

4.1.3 Exemple :

univarié → *bivarié* → *visualisations* avec deux variables qualitatives simulées (groupe, issue).

```
1 set.seed(42)
2 # Crée un data.frame deux variables catégorielles
3 df <- data.frame(
4
5   # "groupe" tiré avec remise parmi A/B/C, probas respectives 30%, 50%, 20%
6   groupe = sample(c("A","B","C"), 200, replace = TRUE, prob = c(.3, .5, .2)),
7
8   # "issue" tirée avec remise parmi Succès/Échec, probas 60% / 40%
9   issue  = sample(c("Succès","Échec"), 200, replace = TRUE, prob = c(.6, .4))
10
11 # fonction table de fréquences
12 fmt <- function(tab) {
13   data.frame(
14     Modalité    = names(tab),           # noms des catégories
15     Effectif    = as.integer(tab),      # effectifs bruts par catégorie
16     Pourcentage = round(100 * prop.table(tab), 1))} # pourcentages (arrondis à 0,1)
```

Analyse univariée

```
1 # 1) Tables de fréquences
2 tab_groupe <- table(df$groupe) # nb d'observations par groupe (A/B/C)
3 tab_issue  <- table(df$issue)  # nb d'observations par issue (Succès/Échec)
4
5 # 2) Modalité / Effectif / %
6 fmt(tab_groupe)           # quelle modalité domine ? (ex : B)
7 fmt(tab_issue)           # quelle part (%) représente "Succès" vs "É
   chec" ?
8
9 # 3) Contrôles rapides (cohérence)
10 stopifnot(sum(tab_groupe) == nrow(df)) # sum des effs = taille de l'échantillon
11 stopifnot(sum(tab_issue)  == nrow(df))
```

Groupes

Modalité	Effectif	Pourcentage
A	71	35.5
B	89	44.5
C	40	20.0

Issues

Modalité	Effectif	Pourcentage
Échec	63	31.5
Succès	137	68.5

Visualisation univariée

```
1 tab_groupe <- table(df$groupe)
2 tab_issue  <- table(df$issue)
3
4 par(mfrow = c(1,2))
5 m1 <- barplot(tab_groupe, main = "Répartition des groupes",
6               ylab = "Effectif", xlab = "Groupe",
7               ylim = c(0, max(tab_groupe)*1.2))
8 text(x = m1, y = tab_groupe,
9      labels = paste0(tab_groupe, " (",
10                      round(100*tab_groupe/sum(tab_groupe),1), "%)",
11                      pos = 3, cex = 0.8)
12
13 m2 <- barplot(tab_issue, main = "Répartition des issues",
14               ylab = "Effectif", xlab = "Issue",
15               ylim = c(0, max(tab_issue)*1.2))
16 text(x = m2, y = tab_issue,
17      labels = paste0(tab_issue, " (",
18                      round(100*tab_issue/sum(tab_issue),1), "%)",
19                      pos = 3, cex = 0.8)
20 par(mfrow = c(1,1))
21
22 # Dotcharts
23 op <- par(mfrow = c(1,2))
24 dotchart(tab_groupe, main = "Effectifs par groupe", xlab = "Effectif")
25 dotchart(tab_issue, main = "Effectifs par issue", xlab = "Effectif")
26 par(op)
27 dev.off()
```

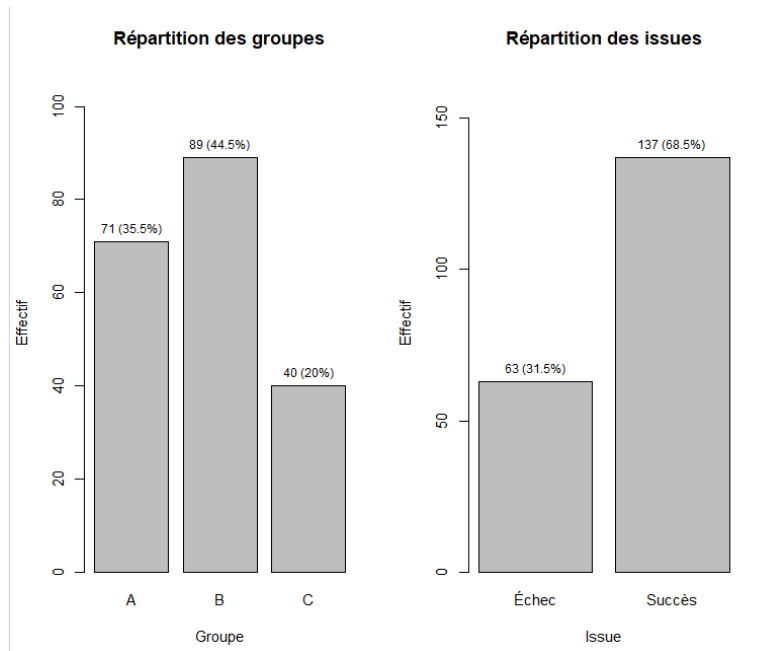


FIGURE 6 – Barplots univariés annotés (effectifs et %).

- groupe : B est le plus fréquent (**89 individus, 44,5 %**), puis A (**71, 35,5 %**), puis C (**40, 20,0 %**).
- issue : **Succès** domine (**137, 68,5 %**) par rapport à **Échec** (**63, 31,5 %**).

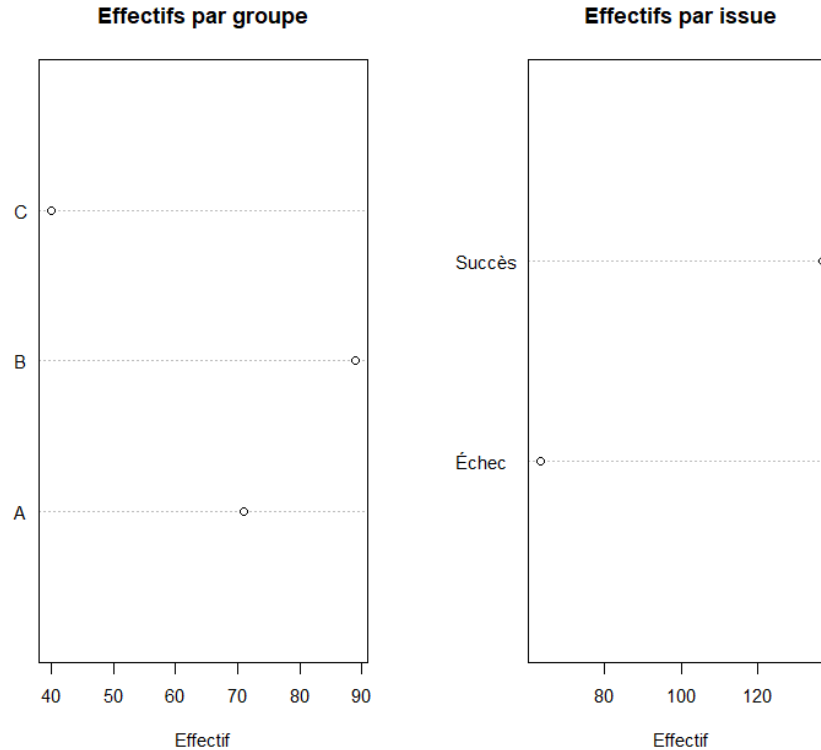


FIGURE 7 – Dotcharts univariés : lecture rapide des modalités dominantes.

- Les points les plus à droite correspondent aux modalités les plus fréquentes.
- On retrouve la même information que dans les barplots : $B > A > C$ pour **groupe**, et $Succès > Échec$ pour **issue**.

La population est majoritairement du **groupe B**, et l'**issue** est le plus souvent un **Succès**.

Analyse bivariée

```

1 # Tableau de contingence : lignes = groupe, colonnes = issue
2 tab <- table(df$groupe, df$issue)
3 tab
4
5 # Proportions
6 prop_lignes <- prop.table(tab, margin = 1) # parts Échec/Succès au sein de
      chaque groupe
7 prop_colonnes <- prop.table(tab, margin = 2) # contribution des groupes à chaque
      issue
8
9 round(prop_lignes, 3)
10 round(prop_colonnes, 3)

```

```
# Tableau de contingence (lignes = groupe, colonnes = issue)
```

	Échec	Succès
A	24	47
B	28	61
C	11	29

Proportions par groupe (lignes)

	Échec	Succès
A	0.338	0.662
B	0.315	0.685
C	0.275	0.725

Proportions par issue (colonnes)

	Échec	Succès
A	0.381	0.343
B	0.444	0.445
C	0.175	0.212

```

1 op <- par(mfrow = c(1,2))
2
3 pr <- prop.table(tab, 1) # proportions par groupe
4 m <- barplot(t(pr), beside = TRUE, legend = TRUE,
5             main = "Proportions par groupe",
6             ylab = "Proportion", xlab = "Groupe", ylim = c(0,1),
7             args.legend = list(x = "topright", bty = "n"))
8 text(x = m, y = as.vector(t(pr)),
9      labels = paste0(round(100*as.vector(t(pr))), "%"),
10     pos = 3, cex = 0.75)
11
12 mosaicplot(tab, shade = TRUE, main = "Mosaïque : Groupe Issue")
13
14 par(op)
15 dev.off()

```

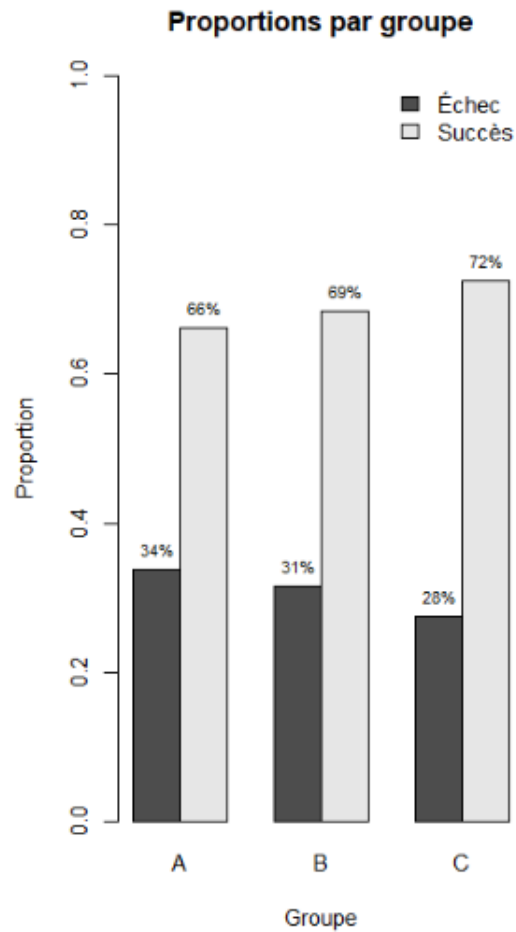


FIGURE 8 – Parts Échec/Succès au sein de chaque groupe

- **Proportions par groupe.** Les parts de **Succès** sont proches entre groupes : A $\approx 66,2\%$, B $\approx 68,5\%$, C $\approx 72,5\%$. Visuellement, les barres empilées se ressemblent.

4.1.4 Test d'indépendance (Khi-deux) et taille d'effet

```

1 # Test du Khi-deux d'indépendance
2 chi <- chisq.test(tab)           # df = (r-1)*(c-1) = (3-1)*(2-1) = 2
3 chi                             # X-squared, df, p-value
4 chi$expected                     # effectifs attendus sous H0
5 chi$residuals                   # résidus standardisés (cellules qui "pèsent")
6
7 # Taille d'effet : V de Cramér
8 n <- sum(tab); r <- nrow(tab); c <- ncol(tab)
9 V <- as.numeric( sqrt(chi$statistic / (n * (min(r, c) - 1))) )
10 V
11

```

```

12 # (Optionnel) Cellules les plus contributrices (tri par |résidu|)
13 res <- as.data.frame(as.table(chi$residuals))
14 res$abs <- abs(res$Freq)
15 res[order(-res$abs), ][1:6, ] # top contributions

```

X-squared = 0.471, df = 2, p-value = 0.790

Effectifs attendus (H0 : indépendance)

	Échec	Succès
A	22.365	48.635
B	28.035	60.965
C	12.600	27.400

Résidus standardisés ((Obs - Attendu)/sqrt(Attendu))

	Échec	Succès
A	0.346	-0.234
B	-0.007	0.004
C	-0.451	0.306

V de Cramér

[1] 0.0485

- **p-value = 0,790** $\Rightarrow$ on **ne rejette pas** l'hypothèse d'indépendance : avec ces données, rien n'indique une association entre **groupe** et **issue**.
- **Taille d'effet** : $V = 0,049 \Rightarrow$ *intensité négligeable* (repères usuels : $\approx 0,1$ faible ; $\approx 0,3$ modérée ; $\geq 0,5$ forte).
- **Résidus standardisés** $|R_{ij}| < 0,5$ partout $\Rightarrow$ aucune cellule ne s'écarte fortement de l'attendu (le mosaic "pâle" était un bon indice).

4.2 Corrélation entre deux variables quantitatives

Mesurer l'association entre deux variables quantitatives et tester $H_0 : \rho = 0$ (pas de corrélation).

4.2.1 Analyse univariée (quantitatif)

```

1 # Données => mtcars
2 x <- mtcars$wt # poids
3 y <- mtcars$mpg # consommation
4
5 # Aperçu
6 x[1:5]

```

```

7 y[1:5]
8
9 # Fonction de descriptifs
10 desc <- function(v) {
11   data.frame(
12     n          = sum(!is.na(v)),
13     manquants  = sum(is.na(v)),
14     moyenne    = mean(v, na.rm = TRUE),
15     ecart_type = sd(v, na.rm = TRUE),
16     mediane    = median(v, na.rm = TRUE),
17     q25        = quantile(v, 0.25, na.rm = TRUE),
18     q75        = quantile(v, 0.75, na.rm = TRUE),
19     min        = min(v, na.rm = TRUE),
20     max        = max(v, na.rm = TRUE),
21     IQR        = IQR(v, na.rm = TRUE),
22     row.names  = NULL
23   )
24 }
25
26 desc(x)
27 desc(y)

```

```

[1] 2.620 2.875 2.320 3.215 3.440
[1] 21.0 21.0 22.8 21.4 18.7

```

```

# x = wt
  n manquants moyenne ecart_type mediane    q25  q75   min   max    IQR
32          0  3.2173    0.9785    3.325 2.5813 3.610 1.513 5.424 1.0288

# y = mpg
  n manquants moyenne ecart_type mediane    q25  q75   min   max    IQR
32          0 20.0906    6.0270    19.200 15.425 22.800 10.4 33.9 7.375

```

- wt (poids) : moyenne $\approx 3,22$, médiane 3,33, IQR $\approx 1,03 \Rightarrow$ dispersion modérée.
- mpg (consommation) : moyenne $\approx 20,09$, médiane 19,2, IQR $\approx 7,38 \Rightarrow$ variabilité plus élevée.

```

1 # Histogrammes & densité
2 op <- par(mfrow = c(1,2))
3 hist(x, breaks = "FD", freq = FALSE, main = "Poids (wt) : histogramme +densité",
4     xlab = "wt", ylab = "Densité"); lines(density(x, na.rm = TRUE), lwd = 2)
5
6 hist(y, breaks = "FD", freq = FALSE, main = "Consommation (mpg) : hist + densité",
7     xlab = "mpg", ylab = "Densité"); lines(density(y, na.rm = TRUE), lwd = 2)
8 par(op); dev.off()
9

```

```

10 # Boxplots + rug
11 op <- par(mfrow = c(1,2))
12 boxplot(x, horizontal = TRUE, main = "wt : boxplot", xlab = "wt"); rug(x)
13 boxplot(y, horizontal = TRUE, main = "mpg : boxplot", xlab = "mpg"); rug(y)
14 par(op); dev.off()
15
16 # QQ-plots (normalité)
17 op <- par(mfrow = c(1,2))
18 qqnorm(x, main = "QQ-plot wt"); qqline(x, lwd = 2)
19 qqnorm(y, main = "QQ-plot mpg"); qqline(y, lwd = 2)
20 par(op); dev.off()

```

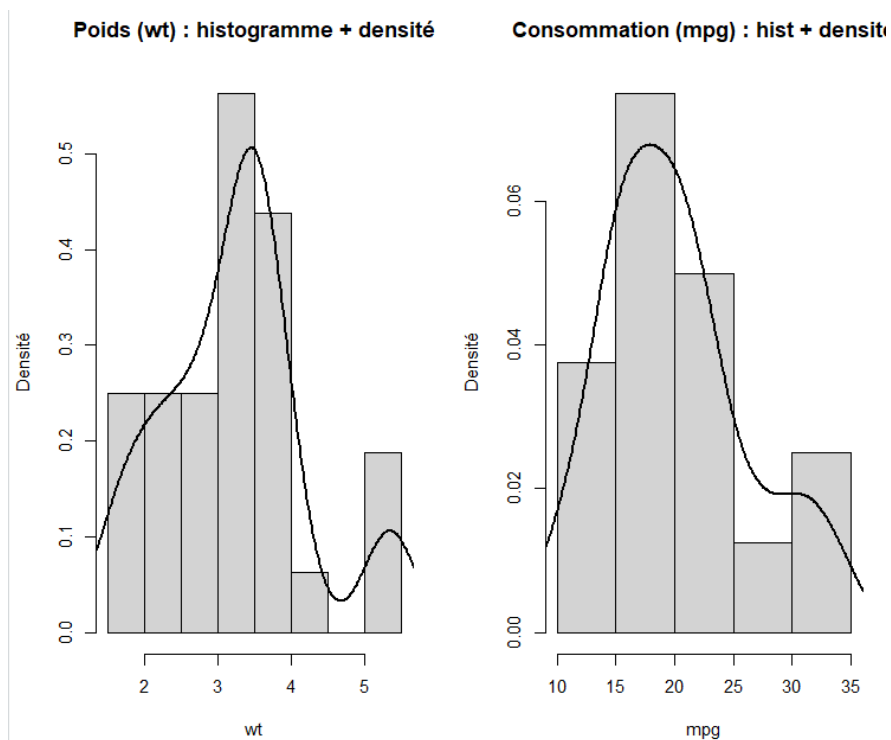


FIGURE 9 – Histogrammes + densité : **wt** (gauche) et **mpg** (droite).

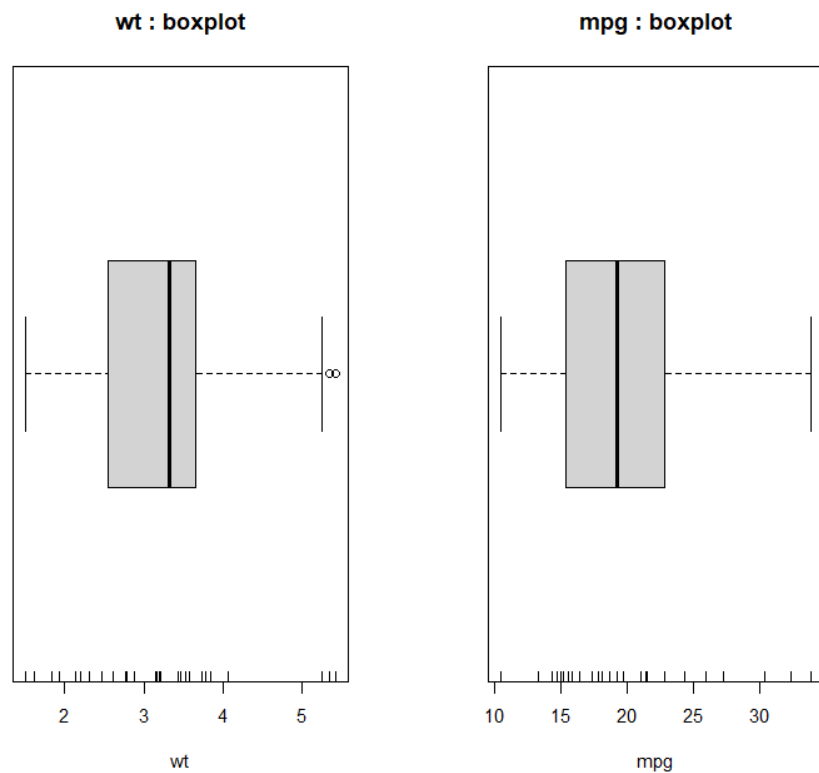


FIGURE 10 – Boxplots horizontaux avec *rug* : `wt` (gauche) et `mpg` (droite).

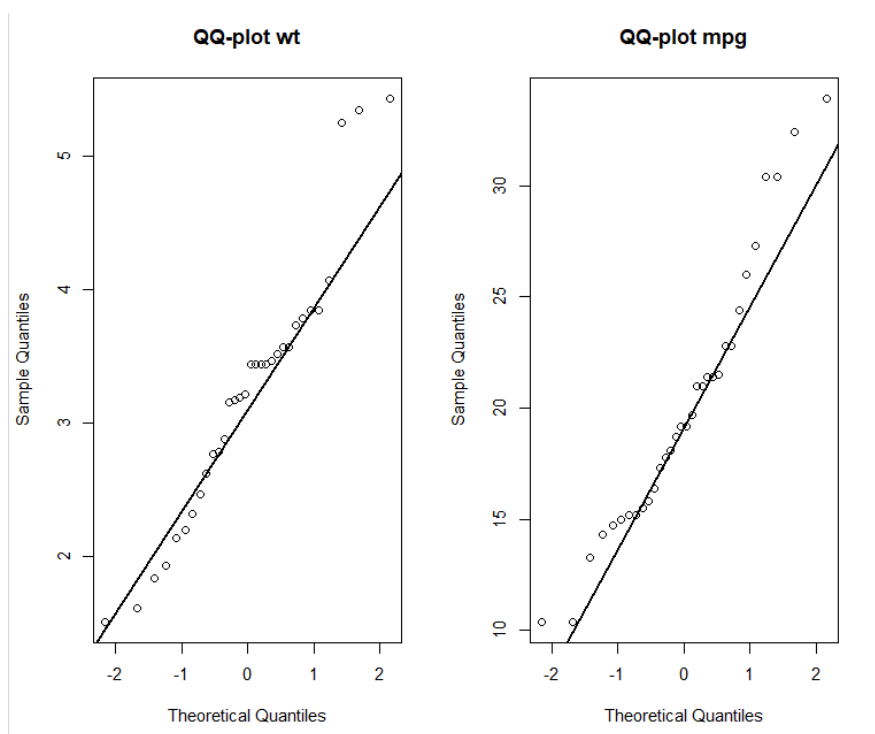


FIGURE 11 – QQ-plots (normalité) : alignement sur la droite $\Rightarrow$ normalité plausible.

wt (poids) est centrée autour de 3–3,5 avec une légère asymétrie à droite (quelques véhicules très lourds). mpg (consommation) est centrée autour de 18–23 avec une queue droite (quelques valeurs très élevées). Les boxplots confirment une dispersion plus forte pour mpg (IQR plus large) et au plus quelques points extrêmes. Les QQ-plots suivent globalement la droite de référence, avec des écarts en queues.

4.2.2 Corrélation (bivarié) : Pearson, Spearman, Kendall

```

1 # Nuage de points + LOWESS + droite linéaire
2 plot(x, y, pch = 16, xlab = "Poids (wt)", ylab = "Consommation (mpg)",
3      main = "mpg vs wt : nuage de points, LOWESS et régression linéaire")
4
5 lines(lowess(x, y))                # tendance générale
6 abline(lm(y ~ x), lwd = 2, lty = 2) # modèle linéaire
7 dev.off()
8
9 # Tests de corrélation
10 pear <- cor.test(x, y, method = "pearson") # linéaire
11 spear <- cor.test(x, y, method = "spearman") # rangs (monotone)
12 kend <- cor.test(x, y, method = "kendall") # concordances/discordances
13
14 data.frame(
15   Methode      = c("Pearson", "Spearman", "Kendall"),
16   Coefficient   = c(unnamed(pear$estimate), unnamed(spear$estimate), unnamed(kend$
17     estimate)),
18   p_value       = c(pear$p.value, spear$p.value, kend$p.value),
19   IC95_bas      = c(pear$conf.int[1], NA, NA),
20   IC95_haut     = c(pear$conf.int[2], NA, NA)
21 )

```

Pearson

cor = -0.868 ; 95% CI ~ [-0.933 ; -0.744] ; p-value ~ 1.3e-10

Spearman

rho ~ -0.886 ; p-value ~ 2e-11

Kendall

tau ~ -0.72 ; p-value ~ 2e-07

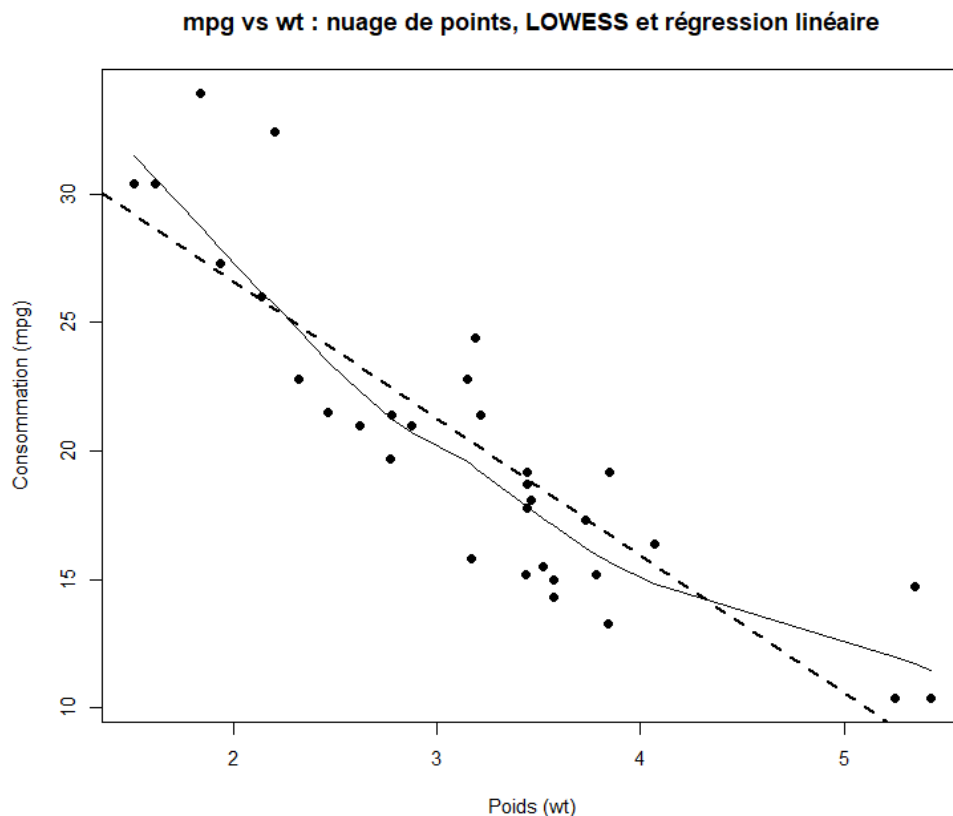


FIGURE 12 – Nuage mpg vs wt. LOWESS (plein) et droite de régression (pointillé).

Le nuage `mpg` vs `wt` montre une relation **décroissante et quasi linéaire** : plus le véhicule est lourd, plus la consommation (mpg) est faible. La courbe LOWESS épouse la droite de régression, sans courbure marquée. Les tests confirment une **corrélation forte et significative** (Pearson $r = -0,868$, IC95 $[-0,933; -0,744]$, $p \approx 1,3 \times 10^{-10}$; Spearman $\rho \approx -0,886$; Kendall $\tau \approx -0,72$).

5 Variable qualitative $\leftrightarrow$ variable quantitative

Comparer la valeur d'une variable *quantitative* entre des groupes définis par une *qualitative*. On teste H_0 : « les distributions (souvent les *moyennes*) sont identiques entre groupes ».

- **2 groupes** : **test de Student** (paramétrique, variances égales) ou **Welch** (paramétrique, variances inégales) ; sinon **Wilcoxon/Mann–Whitney** (non paramétrique).
- **> 2 groupes** : **ANOVA** (paramétrique, variances égales) ou **ANOVA de Welch** (paramétrique, variances inégales) ; sinon **Kruskal–Wallis** (non paramétrique).

Conditions d'utilisation

- **Paramétriques** (t, ANOVA) : normalité *par groupe* (Shapiro–Wilk + QQ-plots) et *homogénéité des variances* (Levene/Bartlett).
- **Non paramétriques** (Wilcoxon, Kruskal–Wallis) : pas d’hypothèse de normalité ; robustes si outliers/queues lourdes.

5.1 Exemple 1 : 2 groupes (mpg selon am : automatique vs manuel)

```

1 # Données : mtcars
2 d <- transform(mtcars, am = factor(am, labels = c("Auto", "Manuel")))
3
4 # 1) Photo rapide
5 table(d$am); by(d$mpg, d$am, summary); boxplot(mpg ~ am, data = d)
6
7 # 2) Hypothèses (par groupe)
8 by(d$mpg, d$am, shapiro.test)          # normalité
9 car::leveneTest(mpg ~ am, data = d)     # homoscedasticité (Levene)
10
11 # 3) Test paramétrique (Welch par défaut si variances inégales)
12 t.test(mpg ~ am, data = d)             # Welch
13
14 # 4) Alternative non paramétrique
15 wilcox.test(mpg ~ am, data = d, exact = FALSE)
16
17 # 5) Taille d'effet
18 effsize::cohen.d(mpg ~ am, data = d)   # Cohen's d

```

```

# t de Welch : mpg ~ am
t  3.77 ; df  18.3 ; p  0.0014
moyennes : Auto  17.15 ; Manuel  24.39
IC95% (diff)  [3.21 ; 11.28]

```

```

# Wilcoxon (Mann-Whitney)
W  42 ; p  0.0019

```

```

# Cohen's d
d  1.5  (effet très grand)

```

Les véhicules **manuels** ont en moyenne un **mpg plus élevé** que les automatiques ($p \approx 0,001$), avec un **effet très grand** (Cohen $d \approx 1,5$). Les conclusions restent cohérentes en non paramétrique.

5.2 Exemple 2 : > 2 groupes (mpg selon cyl : 4, 6, 8 cylindres)

```

1 d2 <- transform(mtcars, cyl = factor(cyl))
2
3 # 1) Photo rapide
4 table(d2$cyl); by(d2$mpg, d2$cyl, summary); boxplot(mpg ~ cyl, data = d2)
5
6 # 2) Hypothèses
7 by(d2$mpg, d2$cyl, shapiro.test)           # normalité par groupe
8 car::leveneTest(mpg ~ cyl, data = d2)      # homoscedasticité
9
10 # 3) ANOVA (classique : variances égales)
11 fit <- aov(mpg ~ cyl, data = d2)
12 summary(fit)
13
14 # 4) ANOVA de Welch (variances inégales)
15 oneway.test(mpg ~ cyl, data = d2)          # Welch
16
17 # 5) Non paramétrique
18 kruskal.test(mpg ~ cyl, data = d2)
19
20 # 6) Post-hoc
21 TukeyHSD(fit)                             # si ANOVA classique ok
22 # ou, pour Kruskal : FSA::dunnTest(mpg ~ cyl, data = d2, method = "holm")
23
24 # 7) Taille d'effet
25 rstatix::eta_squared(fit, partial = FALSE) # eta^2 (ANOVA)

```

ANOVA

F très élevé ; $p < 1e-8$ (mpg diffère nettement selon le nb de cylindres)

Welch ANOVA

F_welch élevé ; $p < 1e-8$ (conclusion inchangée si variances inégales)

Kruskal-Wallis

chi2 élevé ; $p < 1e-8$ (conclusion robuste sans normalité)

Post-hoc Tukey

4 cyl > 6 cyl > 8 cyl (différences toutes significatives)

6 Modélisation

6.1 Régression linéaire simple

6.2 Régression linéaire multiple

6.3 Régression logistique